False Sharing and Scalability in Multicore Processors: Impact of Cache-Line Data Layout on Parallel Performance

Walter Alfaro, Allan Arrieta

***Abstract*—Modern multicore processors keep private caches coherent at the granularity of a cache line, typically 64 bytes, rather than individual variables. When threads running on different cores write to independent variables that happen to reside in the same cache line, the coherence protocol forces the line to migrate repeatedly between cores, even though no data is logically shared. This phenomenon, known as false sharing, generates invalidation traffic and cache-to-cache transfers that can severely limit the scalability of parallel programs and is a recurring source of performance regressions in large systems such as the Linux kernel. This paper investigates how the distribution of variables across cache lines affects the performance of a parallel program on a hybrid x86-64 Intel Core i7-1260P (Alder Lake) processor, which combines performance and efficiency cores. We study a microbenchmark in which each thread, pinned to a dedicated core, updates its own independent counter, comparing a layout where the counters are stored contiguously against layouts in which they are separated through alignment and padding, as well as per-thread accumulation. The analysis relates the observed execution time and scalability with the number of threads to the behavior of the MESI-family coherence protocol, and uses hardware performance counters and the Linux `perf c2c` tool to identify contended cache lines through HITM (load hit in a modified line) events. In addition, the memory and cache-capacity overhead introduced by padding is quantified, together with the mitigation strategies documented by the Linux kernel community, in order to characterize the trade-off between coherence-traffic reduction and memory footprint and to derive practical guidelines for cache-aware data layout.**

***Index Terms*—False sharing, cache coherence, MESI, multicore processors, cache line, padding, performance analysis, perf c2c, Linux kernel.**

I. Introduction

PARALLEL programs are expected to run faster as more cores are added. A well-known obstacle to this expectation is *false sharing*: threads that never touch each other's data can still slow each other down when their data happen to share a cache line [1], [2]. Because the effect depends on memory layout rather than on the algorithm, it is invisible in the source code and is usually discovered only through profiling [3], [4].

[TODO: 1 parrafo: por que importa hoy (kernel, servidores, laptops hibridas). Citar la doc del kernel como fuente del problema, no como prueba de magnitud.]

Most of the guidance on false sharing comes from a small group of sources: processor vendors, the developers of the detection tools, and research prototypes evaluated by their own authors. Rather than taking that guidance at face value, this paper treats each recommendation as a claim to be tested. We pose three questions:

1) **RQ1:** How much does placing independent counters in the same cache line degrade performance and scalability on a contemporary hybrid laptop processor?
2) **RQ2:** Which commonly cited claims about false sharing (magnitude of the slowdown, need for 128-byte padding, reliability of HITM-based detection) hold under our model and measurements, and which depend on the context in which their sources were produced?
3) **RQ3:** What does mitigation cost in memory footprint, and when is that cost worth paying?

[TODO: Contribuciones: (1) simulador MESI propio y abierto; (2) mediciones en i7-1260P con perf c2c; (3) contraste simulacion vs. hardware; (4) lectura critica de las fuentes. Ultimo parrafo: organizacion del paper.]

II. Background

A. Coherence at Cache-Line Granularity

Private caches in a multicore processor are kept coherent by a hardware protocol. The unit tracked by the protocol is the cache line (64 bytes on current x86-64 processors [5]), not the individual variable. Consequently, two variables that reside in the same line are treated by the hardware as a single shared object [6], [7].

B. The MESI Protocol

[TODO: Describir los 4 estados (M, E, S, I), las transacciones BusRd, BusRdX y BusUpgr, y una tabla o diagrama de transiciones. Fuentes: Hennessy–Patterson y Nagarajan et al. Aclarar que Intel usa una variante (MESIF) y que el detalle exacto no es publico.]

C. True Sharing vs. False Sharing

[TODO: Definicion formal (Torrellas et al.): un fallo es de false sharing si la palabra que se invalida no es la que se usa. Condiciones: varios nucleos, misma linea, al menos una escritura [2]. Figura: 4 contadores de 8 B en una linea de 64 B.]

D. Detecting False Sharing with `perf c2c`

[TODO: Que es un HITM (load que encuentra la linea en estado M en otra cache), como perf c2c lo muestrea y que reporta (lineas, offsets, nodos/CPUs) [3], [8]. Anotar desde ya la limitacion: es muestreo y se basa en loads.]

TABLE I
CRITICAL READING OF THE MAIN SOURCES

Source	Context / possible bias	How we test it
Kernel docs [2]	Written by an Intel engineer; cases from large servers	Magnitude on a 4-P-core laptop
Mario [3]	Co-author of the tool he documents	Detection under few or store-only accesses
Intel manual [5]	Vendor document; claims rarely quantified	64 B vs. 128 B padding
Drepper [9]	2007 hardware	Whether costs still hold
Torrellas [1]	Simulated 1990s bus machines	Line-size sweep in our simulator
SHERIFF [4]	Authors evaluate their own tool	**[TODO: revisar trabajos posteriores]**

III. RELATED WORK AND CRITICAL REVIEW OF SOURCES

A. What the Sources Claim

[TODO: Resumen breve de cada fuente: Torrellas 1994 (false sharing vs. localidad espacial y tamano de linea), Liu y Berger 2011 (SHERIFF), doc del kernel (patrones y mitigaciones), Mario 2016 (perf c2c), manual de Intel (recomendaciones de layout y prefetcher), Drepper 2007 (costo de coherencia).]

B. Who Wrote Them, and Under What Conditions

[TODO: Para cada fuente, contexto y posible sesgo. Verificar cada punto contra la fuente antes de afirmarlo.]

IV. METHODOLOGY

A. Workload

Each of N threads increments its own 64-bit counter I times. No counter is ever read or written by another thread, so any coherence traffic between threads is, by construction, false sharing. Four data layouts are compared:

- **packed:** counters stored contiguously (8 B apart), so up to eight counters share one 64-byte line;
- **padded:** counters separated by a stride s (64 or 128 B), so each occupies its own line;
- **local:** each thread accumulates in a private variable and writes the shared array once at the end;
- **shared:** all threads increment the *same* counter (true sharing), used as a reference.

B. Trace-Driven MESI Simulator

We implemented a MESI coherence simulator in C (available in the project repository, `sim/mesi_sim.c`). Its model is deliberately simple so that every result can be traced to a protocol event:

- N cores, each with a private set-associative LRU cache;
- a single, serialized snooping bus carrying BusRd, BusRdX and BusUpgr transactions;
- an atomic increment modeled as a read-for-ownership (BusRdX) followed by a write, and, alternatively, as a separate load and store;

TABLE II
SIMULATOR LATENCY PARAMETERS (ESTIMATES, IN CYCLES)

Event	Cycles
Private cache hit	4
Data from LLC / clean copy	40
Data from a modified line in another core (HITM)	70
Main memory	200
Upgrade (S→M, invalidation only)	30

- a per-core clock; at each step the core with the lowest clock issues its next access, so contention emerges from timing instead of from a fixed interleaving;
- an optional *adjacent-line prefetch* that, on a miss, also fetches the other line of the aligned 128-byte pair.

The simulator counts hits, misses, bus transactions, invalidations, cache-to-cache transfers from a modified line (the analogue of a HITM), write-backs, and the memory occupied by the counters. Latencies are model parameters, not measurements (Table II); their values are estimates and a sensitivity sweep is reported.

[TODO: Justificar los valores de latencia con alguna fuente o con la medicion propia (p. ej. derivar la latencia HITM del experimento real) y aclarar que el bus serializado es una simplificacion: el i7-1260P usa un ring y una L3 compartida.]

Four sweeps are run (`sim/sweep.sh`): (i) scaling with N for each layout; (ii) line size versus counter stride; (iii) adjacent-line prefetch with 64-byte and 128-byte padding; and (iv) sensitivity to the HITM latency.

C. Hardware Experiment

[TODO: Plataforma: Intel Core i7-1260P, 4 P-cores (HT) + 8 E-cores, L3 18 MiB, Linux [TODO: kernel]. Hilos fijados con `taskset -c 0,2,4,6` (un hilo por nucleo fisico). Compilador y flags. Numero de repeticiones, media y desviacion. Grabacion con `perf c2c record` por layout. Opcional: E-cores de un mismo cluster (L2 compartida).]

V. RESULTS

A. Simulation

[TODO: Figura: ciclos por incremento vs. N para los cuatro layouts (`results/sim/scaling.csv`). Tabla: linea vs. stride (`line_vs_stride.csv`). Resultado del prefetch (`prefetch.csv`). Sensibilidad (`hitm_sensitivity.csv`).]

B. Measurements on the i7-1260P

[TODO: Tiempo vs. hilos por layout (media $\pm$ desviacion), speedup, HITM de perf c2c por layout con los offsets de cada hilo.]

C. Simulation vs. Hardware

[TODO: Donde coinciden y donde no. Las diferencias son material para la discusion.]

VI. Discussion

A. Is False Sharing as Costly as True Sharing?

[TODO: En el modelo, packed y shared generan el mismo trafico. Comprobar en hardware y discutir que implica: para el hardware no existe diferencia.]

B. Does 128-Byte Padding Matter?

[TODO: En el modelo, el prefetch de la linea adyacente solo provoca un evento aislado cuando los contadores ya estan en estado estable: la recomendacion de 128 B dependeria de que haya fallos repetidos. Contrastar con el manual de Intel y con la medicion.]

C. Generalizing from Servers to a Hybrid Laptop

[TODO: Las fuentes describen servidores con muchos nucleos; aqui hay 4 P-cores y una L3 compartida. Que cambia y que no.]

D. Limits of HITM-Based Detection

[TODO: perf c2c muestrea loads; que pasa con patrones dominados por stores.]

E. The Cost of Mitigation

[TODO: 8 B vs. 64 B vs. 128 B por contador: 8x o 16x mas memoria, mas lineas de cache y presion de TLB. Cuando no vale la pena (contadores poco escritos).]

F. Threats to Validity

[TODO: Simplificaciones del simulador, frecuencia variable/turbo, scheduler, microbenchmark como peor caso.]

VII. Conclusions

[TODO: Responder RQ1–RQ3 en un parrafo cada una y dar guias practicas.]

References

[1] J. Torrellas, M. S. Lam, and J. L. Hennessy, "False sharing and spatial locality in multiprocessor caches," *IEEE Transactions on Computers*, vol. 43, no. 6, pp. 651–663, Jun. 1994.

[2] F. Tang, "False sharing," The Linux Kernel Documentation, Kernel Hacking Guides, 2023, accessed: Nov. 5, 2026. [Online]. Available: https://kernel.org/doc/html/latest/kernel-hacking/false-sharing.html

[3] J. Mario, "C2C – false sharing detection in Linux perf," Sep. 2016, accessed: Nov. 5, 2026. [Online]. Available: https://joemario.github.io/blog/2016/09/01/c2c-blog/

[4] T. Liu and E. D. Berger, "SHERIFF: Precise detection and automatic mitigation of false sharing," in *Proc. ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, Portland, OR, USA, 2011, pp. 3–18.

[5] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual, Volume 1*, May 2024, rev. 051. Accessed: Nov. 5, 2026. [Online]. Available: https://www.intel.com/content/www/us/en/content-details/814198/intel-64-and-ia-32-architectures-optimization-reference-manual-volume-1.html

[6] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[7] V. Nagarajan, D. J. Sorin, M. D. Hill, and D. A. Wood, *A Primer on Memory Consistency and Cache Coherence*, 2nd ed., ser. Synthesis Lectures on Computer Architecture. Springer, 2020.

[8] Linux perf developers, "perf-c2c(1) – shared data C2C/HITM analyzer," Linux manual page, accessed: Nov. 5, 2026. [Online]. Available: https://man7.org/linux/man-pages/man1/perf-c2c.1.html

[9] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., Tech. Rep., Nov. 2007. [Online]. Available: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf